

client.py

```
1  from PIL import Image, ImageTk
2  from tkinter import messagebox
3  from typing import Callable
4  from paint import Paint
5  from tkinter import ttk
6  import tkinter as tk
7  from main import app
8  import subprocess
9  import threading
10 import uvicorn
11 import sys
12 import re
13
14 class App(tk.Tk):
15     def __init__(self):
16         super().__init__()
17         self.title("Paint - Multiplayer")
18         self.icon = tk.PhotoImage(file='assets/icon.png')
19         self.iconphoto(False, self.icon)
20
21         self.resizable = True
22         self.geometry("800x600")
23
24         self.mainMenu = MainMenu(self)
25
26         self.mainMenu.pack(expand = True, fill = "both")
27
28
29         self.mainloop()
30
31     def save(self):
32         if self.paintScreen:
33             self.paintScreen.save()
34
35     def launchPaint(self, network: bool, wsUrl: str = "ws://localhost:8000"):
36         self.mainMenu.destroy()
37
38         self.paintScreen = Paint(master = self, network = network, wsUrl = wsUrl)
39
40         self.paintScreen.pack(expand = True, fill = "both")
41
42         self.bind("x", lambda _: self.paintScreen.colorBar.swapPrimarySecondary())
43
44         self.configure(menu = MenuBar(self, network))
45
46     def copyTunnelUrl(self):
47         if self.tunnelUrl:
```

```

48         self.clipboard_clear()
49         self.clipboard_append(self.tunnelUrl)
50         self.update()
51
52     def runUvicorn(self):
53         self.serverProcess = subprocess.Popen([
54             sys.executable,
55             "-m", "uvicorn",
56             "main:app",
57             "--host", "127.0.0.1",
58             "--port", "8000"
59         ])
60
61     def launchServer(self, on_tunnel_ready: Callable[[str], None]) -> None:
62
63         # Server
64         self.runUvicorn()
65
66         # Tunnel
67
68         self.tunnelProcess = subprocess.Popen(
69             [
70                 "./cloudflared.exe", "tunnel", "--url", "http://localhost:8000", "--
protocol", "http2"],
71             stdout = subprocess.PIPE,
72             stderr = subprocess.STDOUT,
73             text = True
74         )
75
76         self.tunnelUrl = None
77
78         # Stdout Thread
79         def readOutput():
80             for line in self.tunnelProcess.stdout: # type: ignore
81                 match = re.search(r"https://.*\.trycloudflare\.com", line)
82                 if match:
83                     self.tunnelUrl = match.group(0).replace("https://", "wss://")
84                     break
85
86         threading.Thread(target = readOutput, daemon = True).start()
87
88         # Loader
89         self.loadingWindow = tk.Toplevel(self)
90         self.loadingWindow.title("Starting server...")
91
92         self.loader = ttk.Progressbar(self.loadingWindow, mode = "indeterminate",
length = 300)
93         self.loader.pack(padx = 20, pady = 20)
94         self.loader.start()

```

```

95         # Polling
96
97     def poll():
98         if self.tunnelUrl:
99             self.loader.stop()
100             self.loadingWindow.destroy()
101             on_tunnel_ready(self.tunnelUrl)
102         else:
103             self.after(100, poll)
104
105     poll()
106
107
108 class MenuBar(tk.Menu):
109     def __init__(self, master: App, network: bool, *args, **kwargs):
110         super().__init__(*args, **kwargs)
111
112         self.configure(
113             background = "darkgray",
114             relief = "raised",
115             border = 2
116         )
117
118         # Sub Menus
119         self.fileMenu = tk.Menu(self, tearoff = False)
120         self.fileMenu.add_command(label = "New")
121         self.fileMenu.add_command(label = "Save", command = master.save)
122         if network:
123             self.fileMenu.add_command(label = "Copy join link.", command =
master.copyTunnelUrl)
124         self.fileMenu.add_separator()
125         self.fileMenu.add_command(label = "Exit", command = self.master.quit)
126
127         # Adding Submenus to Menubar
128         self.add_cascade(label = "File", menu = self.fileMenu)
129
130 class MainMenu(tk.Frame):
131     def __init__(self, master: App, *args, **kwargs):
132         super().__init__(master = master)
133
134         self.app: App = master
135
136         self.configure(background = "#FEFEFE")
137
138         self.logoSize = (430, 255)
139         self.logoImage = ImageTk.PhotoImage(Image.open("assets/
logo.png").resize((self.logoSize[0], self.logoSize[1])))
140
141         self.logo = tk.Label(

```

```

142         self,
143         image = self.logoImage,
144         width = self.logoSize[0],
145         height = self.logoSize[1]
146     )
147
148     self.logo.pack(expand = True)
149
150     self.buttonWrapper = tk.Frame(self, background = "#FEFEFE")
151
152     self.singlePlayer = tk.Button(
153         self.buttonWrapper,
154         text = "Single Player",
155         command = lambda: self.app.launchPaint(False)
156     )
157
158     self.singlePlayer.pack(expand = True, pady = 5)
159
160     self.multiPlayer = tk.Button(
161         self.buttonWrapper,
162         text = "Multi Player",
163         command = self.networkScreen
164     )
165
166     self.multiPlayer.pack(expand = True, pady = 5)
167
168     self.quitButton = tk.Button(
169         self.buttonWrapper,
170         text = "Quit",
171         command = self.master.quit
172     )
173
174     self.quitButton.pack(expand = True, pady = 5)
175
176     self.buttonWrapper.pack(expand = True)
177
178     def networkScreen(self):
179         for child in self.wininfo_children():
180             child.destroy()
181
182         self.hostRoomButton = tk.Button(self, text = "New Room", command =
self.hostRoom)
183         self.hostRoomButton.pack(expand = True, anchor = "s")
184
185         self.orLabel = tk.Label(self, text = "OR", font = ("TKDefaultFont", 30))
186         self.orLabel.pack(pady = 20)
187
188         self.entryLabel = tk.Label(self, text = "Existing room websocket url:")
189         self.entryLabel.pack()

```

```
190         self.urlEntry = tk.Entry(self, width = 60)
191         self.urlEntry.pack()
192         self.connectButton = tk.Button(self, text = "Connect", command =
self.connect)
193         self.connectButton.pack(expand = True, anchor = "n")
194
195     def connect(self):
196         self.app.launchPaint(wsUrl = self.urlEntry.get(), network = True)
197         self.destroy()
198
199     def hostRoom(self):
200         self.app.launchServer(self.tunnelReady)
201
202     def tunnelReady(self, tunnelUrl: str):
203         self.showUrl = messagebox.showinfo("Websocket URL", f"Your Url:
{tunnelUrl}")
204         print(tunnelUrl)
205         self.app.launchPaint(network = True, wsUrl = tunnelUrl)
206         self.destroy()
207
208 if __name__ == "__main__":
209     App()
```

paint.py

```
1  from __future__ import annotations
2  from typing import TYPE_CHECKING
3  from network import NetworkHandler
4  from dataclasses import asdict
5  from tkinter import filedialog
6  from PIL import Image, ImageTk
7  from colorbar import ColorBar
8  from toolbar import ToolBar
9  from point import Point
10 import tkinter as tk
11 import numpy as np
12 import actions
13
14 if TYPE_CHECKING:
15     from tools import Tool
16     from client import App
17
18 class Paint(tk.Frame):
19     def __init__(self, master: App, wsUrl: str, network = False, *args, **kwargs):
20         super().__init__(master = master, *args, **kwargs)
21
22         self.configure(background = "gray")
23
24         self.master = master
25
26         self.width = 600
27         self.height = 600
28
29         self.pixel_size = 4
30
31         self.undoStack = []
32         self.redoStack = []
33
34         self.lastPos: Point = Point(0, 0)
35         self.currentStroke = []
36
37         # Toolbar + Colorbar
38
39         self.colorBar = ColorBar(self, self.setColor)
40         self.colorBar.pack(side = "bottom", fill = "x")
41
42         self.toolBar = ToolBar(self, self.setTool)
43         self.toolBar.pack(side = "left", fill = "y")
44
45         self.currentTool = self.toolBar.getTool()
46         self.currentColor: tuple[int, int, int] = self.colorBar.getColor()
47
```

```

48         # Canvas Buffer and Image
49
50         self.canvas = tk.Canvas(self,
51             width = self.width,
52             height = self.height,
53             borderwidth = 0,
54             highlightthickness = 0
55         )
56
57         self.canvas.pack(expand = True, anchor = "nw")
58
59         self.buffer = np.full((int(self.height / self.pixel_size), int(self.width /
self.pixel_size), 3), 255, dtype = np.uint8)
60         self.tempBuffer = np.zeros((int(self.height / self.pixel_size),
int(self.width / self.pixel_size), 3), dtype = np.uint8)
61         self.tempMask = np.zeros((int(self.height / self.pixel_size),
int(self.width / self.pixel_size)), dtype = bool)
62
63         self.image = self.canvas.create_image(0, 0, anchor = "nw")
64
65         self.render()
66
67         # Canvas Events
68
69         self.canvas.bind("<B1-Motion>", self.mouseDrag)
70         self.canvas.bind("<ButtonRelease-1>", self.mouseUp)
71         self.canvas.bind("<ButtonPress-1>", self.mouseDown)
72         self.canvas.bind("<Motion>", self.mouseMove)
73         self.master.bind("<Return>", lambda _: self.keyEnter())
74
75         # Network
76         self.networkHandler = NetworkHandler(self, wsUrl)
77
78         if network:
79             self.networkHandler.run()
80
81         self.master.protocol("WM_DELETE_WINDOW", self.close)
82
83         def save(self):
84             filePath = filedialog.asksaveasfilename(
85                 filetypes = [('PNG Files', '*.png'), ('JPEG Files', '*.jpg')],
86                 defaultextension = ".png",
87                 title = "Save Image As"
88             )
89
90             image = Image.fromarray(self.buffer)
91             image.save(filePath)
92
93         def close(self):

```

```

94         self.master.destroy()
95         self.networkHandler.stop()
96
97     def setTool(self, tool: Tool):
98         self.currentTool = tool
99
100    def setColor(self, color):
101        self.currentColor = color
102
103    def mouseDrag(self, event):
104        event.x = int(event.x / self.pixel_size)
105        event.y = int(event.y / self.pixel_size)
106        self.currentTool.on_mouse_drag(self, event)
107        self.render()
108
109    def mouseUp(self, event):
110        event.x = int(event.x / self.pixel_size)
111        event.y = int(event.y / self.pixel_size)
112        self.currentTool.on_mouse_up(self, event)
113        self.clearTempBuffer()
114        self.render()
115
116    def mouseDown(self, event):
117        event.x = int(event.x / self.pixel_size)
118        event.y = int(event.y / self.pixel_size)
119        self.currentTool.on_mouse_down(self, event)
120        self.lastPos = Point(event.x, event.y)
121        self.render()
122
123    def mouseMove(self, event):
124        event.x = int(event.x / self.pixel_size)
125        event.y = int(event.y / self.pixel_size)
126        self.currentTool.on_mouse_move(self, event)
127
128    def keyEnter(self):
129        self.currentTool.on_enter(self)
130
131    def clearTempBuffer(self):
132        self.tempBuffer.fill(0)
133        self.tempMask.fill(False)
134
135    def rect(self, start: Point, stop: Point, fill = False, temp = False):
136        if temp:
137            self.clearTempBuffer()
138            actions.rect(self.tempBuffer, start, stop, self.currentColor, fill,
self.tempMask)
139
140        else:
141            actions.rect(self.buffer, start, stop, self.currentColor)

```



```

142
143         self.networkHandler.sendAction({
144             "rect": {
145                 "start": asdict(start),
146                 "stop": asdict(stop),
147                 "color": self.currentColor,
148                 "fill": fill
149             })
150
151
152     def line(self, start: Point, stop: Point, temp = False):
153         if temp:
154             self.clearTempBuffer()
155             actions.line(self.tempBuffer, start, stop, self.currentColor,
self.tempMask)
156
157         else:
158             actions.line(self.buffer, start, stop, self.currentColor)
159
160             self.networkHandler.sendAction({
161                 "line": {
162                     "start": asdict(start),
163                     "stop": asdict(stop),
164                     "color": self.currentColor
165                 }
166             })
167
168     def ellipse(self, start: Point, stop: Point, fill = False, temp = False):
169         if temp:
170             self.clearTempBuffer()
171             actions.ellipse(self.tempBuffer, start, stop, self.currentColor, fill,
self.tempMask)
172
173         else:
174             actions.ellipse(self.buffer, start, stop, self.currentColor, fill)
175
176             self.networkHandler.sendAction({
177                 "ellipse": {
178                     "start": asdict(start),
179                     "stop": asdict(stop),
180                     "color": self.currentColor,
181                     "fill": fill
182                 }
183             })
184
185     def fill(self, point: Point):
186         actions.floodFill(self.buffer, point, self.currentColor)
187
188         self.networkHandler.sendAction({

```

```

189         "fill": {
190             "point": asdict(point),
191             "color": self.currentColor
192         }
193     })
194
195     def onCanvas(self, x, y) -> bool:
196         if 0 <= x * self.pixel_size < self.width and 0 <= y * self.pixel_size <
self.height:
197             return True
198         else:
199             return False
200
201     def render(self):
202         # NumPy -> PIL -> PhotoImage
203
204         # Expand mask into third dimension to make np.where work [x, y, None] =
True | False
205         mask = self.tempMask[..., None]
206
207         pilImage = Image.fromarray(
208             np.where(mask, self.tempBuffer, self.buffer)
209         ).resize(
210             (self.width, self.height),
211             Image.NEAREST # type: ignore
212         )
213
214         self.imageTK = ImageTk.PhotoImage(pilImage)
215
216         self.canvas.itemconfig(self.image, image = self.imageTK)
217
218
219     def runAction(self, data):
220         if "rect" in data:
221             rect = data["rect"]
222             actions.rect(self.buffer, Point(**rect["start"]),
Point(**rect["stop"]), rect["color"], rect["fill"])
223
224         elif "line" in data:
225             line = data["line"]
226             actions.line(self.buffer, Point(**line["start"]),
Point(**line["stop"]), line["color"])
227
228         elif "ellipse" in data:
229             ellipse = data["ellipse"]
230             actions.ellipse(self.buffer, Point(**ellipse["start"]),
Point(**ellipse["stop"]), ellipse["color"], ellipse["fill"])
231
232         elif "fill" in data:
233             fill = data["fill"]

```

```
234         actions.floodFill(self.buffer, Point(**fill["point"]), fill["color"])
235
236     self.render()
```

colorbar.py

```
1 from collections.abc import Callable
2 from tkinter import colorchooser
3 from PIL import Image, ImageTk
4 import tkinter as tk
5
6 class ColorBar(tk.Frame):
7     def __init__(self, master, on_color_change: Callable[[tuple], None], *args,
8         **kwargs):
9         super().__init__(master = master, *args, **kwargs)
10
11         self.callback = on_color_change
12
13         self.configure(
14             background = "lightgray",
15             height = 64,
16             relief = "raised",
17             border = 2,
18         )
19
20         self.pack_propagate(False)
21
22         # Primary + Secondary Colors
23
24         self.wrapper = tk.Frame(
25             master = self,
26             width = 36,
27             height = 36,
28             background = "#DFDFDF",
29             relief = "sunken",
30             border = 2
31         )
32
33         self.primaryColor = "black"
34         self.secondaryColor = "white"
35
36         self.primaryColorFrame = tk.Frame(
37             master = self.wrapper,
38             width = 12,
39             height = 12,
40             background = self.primaryColor,
41             relief = "sunken",
42             border = 1
43         )
44
45         self.secondaryColorFrame = tk.Frame(
46             master = self.wrapper,
```

```

47         height = 12,
48         background = self.secondaryColor,
49         relief = "sunken",
50         border = 1
51     )
52
53     self.secondaryColorFrame.place(relx = 0.6, rely = 0.6, anchor = "center")
54     self.primaryColorFrame.place(relx = 0.4, rely = 0.4, anchor = "center")
55     self.primaryColorFrame.lift()
56
57     self.wrapper.pack(side = "left", padx = 10)
58
59     # Bind to wrapper and children for proper hover and click event
60     for widget in self.wrapper.winfo_children() + [self.wrapper]:
61         widget.bind("<Enter>", lambda _: self.wrapper.configure(background =
"lightgray"))
62         widget.bind("<Leave>", lambda _: self.wrapper.configure(background =
"#DFDFDF"))
63         widget.bind("<Button-1>", lambda _: self.swapPrimarySecondary())
64
65     # Custom color gradient button
66     self.customColor = tk.Canvas(
67         self,
68         width = 32,
69         height = 32,
70         border = 2,
71         relief = "sunken",
72     )
73
74     # Resize with PIL
75     self.gradientButtonImage = Image.open("assets/gradient.png")
76     self.gradientButtonImage = self.gradientButtonImage.resize((48, 48))
77     self.gradientButtonImage = ImageTk.PhotoImage(self.gradientButtonImage)
78
79     self.customColor.create_image(0, 0, anchor = "nw", image =
self.gradientButtonImage)
80
81     self.customColor.pack(
82         anchor = "w",
83         padx = 10,
84         expand = True,
85         side = "right",
86     )
87
88     self.customColor.bind(
89         "<Button-1>",
90         lambda _: self.setPrimaryColor(colorchooser.askcolor()[1])
91     )
92

```

```

93         # Presets
94
95         self.presets = {
96             "columns": 10 ,
97             "rows": 2,
98             "size": 16,
99             "gap": 4,
100            "colors": [
101                ["red", "orange", "yellow", "green", "blue", "purple", "pink",
102                "brown", "black", "gray"],
103                ["darkred", "darkorange", "gold", "darkgreen", "darkblue",
104                "purple2", "deeppink2", "white", "white", "white"]
105            ]
106        }
107
108        for row in self.presets["colors"]:
109            while len(row) < self.presets["columns"]:
110                row.append("")
111
112        self.presetFrame = tk.Frame(
113            master = self,
114            width = self.presets["columns"] * (self.presets["size"] +
115            self.presets["gap"]),
116            height = self.presets["rows"] * (self.presets["size"] +
117            self.presets["gap"]),
118            relief = "sunken",
119            background = "lightgray"
120        )
121
122        self.presetFrame.pack(anchor = "w", side = "right")
123
124        for x in range(self.presets["columns"]):
125            for y in range(self.presets["rows"]):
126                frame = tk.Frame(
127                    master = self.presetFrame,
128                    width = self.presets["size"],
129                    height = self.presets["size"],
130                    relief = "sunken",
131                    border = 2,
132                    background = self.presets["colors"][y][x] or "white",
133                )
134
135                frame.grid(
136                    column = x,
137                    row = y,
138                    padx = self.presets["gap"] // 2,
139                    pady = self.presets["gap"] // 2
140                )
141
142        # Note: lambda for freezing

```

```

139         frame.bind(
140             "<Button-1>",
141             lambda _, x = x, y = y:
142                 self.setPrimaryColor(self.presets["colors"][y][x])
143         )
144
145         frame.bind(
146             "<Shift-Button-1>",
147             lambda _, frame = frame, x = x, y = y:
148                 (
149                     frame.configure(background = self.primaryColor), # Button
bg
150                     self.setPresetColor((x, y))
151                 )
152         )
153
154     def setPrimaryColor(self, color):
155         self.primaryColor = color
156         self.primaryColorFrame.configure(bg = color)
157         self.callback(self.getColor())
158
159     def setSecondaryColor(self, color):
160         self.secondaryColor = color
161         self.secondaryColorFrame.configure(bg = color)
162
163     def setPresetColor(self, location: tuple[int, int]):
164         self.presets["colors"][location[1]][location[0]] = self.primaryColor
165
166     def swapPrimarySecondary(self):
167         primary = self.primaryColor
168         secondary = self.secondaryColor
169
170         self.setPrimaryColor(secondary)
171         self.setSecondaryColor(primary)
172
173     def getColor(self) -> tuple[int, int, int]:
174         # Raw RGB 0..65535 to 0..255
175         # Type checking ignored because i know the value is correct
176         return tuple([color // 256 for color in
self.master.wininfo_rgb(self.primaryColor)]) # type: ignore
177

```

toolbar.py

```
1 from collections.abc import Callable
2 from PIL import Image, ImageTk
3 import tkinter as tk
4 from tools import *
5
6 class Toolbar(tk.Frame):
7     def __init__(self, master, on_tool_change: Callable[[Tool], None], *args,
8         **kwargs):
9         super().__init__(master = master, *args, **kwargs)
10
11         self.configure(
12             background = "lightgray",
13             width = 64,
14             relief = "raised",
15             border = 2
16         )
17
18         self.pack_propagate(False)
19
20         self.callback = on_tool_change
21
22         self.currentTool = "pencil"
23
24         # 16 x 16 icons in a row scales to self.toolIconSize
25         self.toolIconSize = 16
26         self.toolCount = 16
27         self.toolIconSheet = Image.open("assets/tools.png").resize((
28             self.toolIconSize * self.toolCount,
29             self.toolIconSize),
30             Image.NEAREST) # type: ignore
31
32         self.toolIcons = [] # Made so that images aren't garbage collected
33
34         # Names in order of image
35         self.tools = {
36             "freeform-selection": Tool(),
37             "rectangular-selection": Tool(),
38             "eraser": Eraser(),
39             "paint-bucket": Bucket(),
40             "eye-dropper": Eyedropper(),
41             "zoom": Tool(),
42             "pencil": Pencil(),
43             "brush": Tool(),
44             "spray-can": Tool(),
45             "type": Tool(),
46             "line": Line(),
47             "curve": Tool(),
```



```

47         "rectangle-shape": Rectangle(),
48         "custom-shape": CustomShape(),
49         "circle-shape": Ellipse(),
50         "beveled-rectangle-shape": Tool()
51     }
52
53     self.wrapper = tk.Frame(self)
54
55     i = 0 # Icon itterator
56
57     for x in range(2):
58         for y in range(8):
59             frame = tk.Frame(
60                 master = self.wrapper,
61                 relief = "raised",
62                 border = 2,
63                 name = list(self.tools)[i]
64             )
65
66             self.toolIcons.append(
67                 ImageTk.PhotoImage(
68                     self.toolIconSheet.crop((
69                         i * self.toolIconSize,
70                         0,
71                         i * self.toolIconSize + self.toolIconSize,
72                         self.toolIconSize
73                     ))
74                 )
75             )
76
77             # Label for image
78             tk.Label(
79                 frame,
80                 image = self.toolIcons[-1]).pack()
81
82             frame.grid(
83                 column = x,
84                 row = y
85             )
86
87             # Bind click to label and not frame
88             for widget in frame.winfo_children():
89                 widget.bind("<Button-1>", lambda _, name = list(self.tools)[i]:
self.setTool(name))
90
91             i -= 1 # Lol
92
93
94     self.wrapper.pack(anchor = "n", expand = True, pady = 5)

```

```
95
96     # Set relief of current tool initially
97     self.setTool("pencil")
98
99     def setTool(self, toolName: str):
100
101         # Reset style of current
102         self.wrapper.nametowidget(
103             self.currentTool
104         ).configure(
105             relief = "raised"
106         )
107
108         # Set new tool and style
109         self.currentTool = toolName
110
111         self.wrapper.nametowidget(
112             toolName
113         ).configure(
114             relief = "sunken"
115         )
116
117         # Paint callback
118         self.callback(self.tools[self.currentTool])
119
120     def getTool(self) -> Tool:
121         return self.tools[self.currentTool]
122
```

tools.py

```
1  from __future__ import annotations
2  from typing import TYPE_CHECKING
3  from point import Point
4  import numpy as np
5
6  if TYPE_CHECKING:
7      from paint import Paint
8
9  class Tool:
10     def on_mouse_move(self, app: Paint, event):
11         pass
12
13     def on_mouse_down(self, app: Paint, event):
14         pass
15
16     def on_mouse_drag(self, app: Paint, event):
17         pass
18
19     def on_mouse_up(self, app: Paint, event):
20         pass
21
22     def on_enter(self, app: Paint):
23         pass
24
25  class Pencil(Tool):
26     def on_mouse_down(self, app: Paint, event):
27         self.draw(app, event.x, event.y, app.currentColor)
28
29     def on_mouse_drag(self, app: Paint, event):
30         app.line(app.lastPos, Point(event.x, event.y))
31         app.lastPos = Point(event.x, event.y)
32
33     def on_mouse_up(self, app: Paint, event):
34         app.currentStroke.append((event.x, event.y))
35         app.currentStroke = []
36
37     def draw(self, app: Paint, x, y, color):
38         if app.onCanvas(x, y):
39             app.buffer[y, x] = color
40
41  class Bucket(Tool):
42     def on_mouse_down(self, app: Paint, event):
43         app.fill(Point(event.x, event.y))
44
45  class Eraser(Tool):
46     def on_mouse_down(self, app: Paint, event):
47         self.erase(app, event.x, event.y)
```

```

48
49     def on_mouse_drag(self, app: Paint, event):
50         num_steps = max(np.abs(np.array((app.lastPos.x - event.x, app.lastPos.y -
51 event.y)))) + 1
52         points = np.round(np.linspace((app.lastPos.x, app.lastPos.y), (event.x,
53 event.y), num_steps, retstep = False)).astype(int)
54
55         app.lastPos = Point(event.x, event.y)
56
57         for point in points:
58             self.erase(app, point[0], point[1])
59             app.currentStroke.append((int(point[0]), int(point[1])))
60
61     def on_mouse_up(self, app: Paint, event):
62         app.currentStroke.append((event.x, event.y))
63         app.currentStroke = []
64
65     def erase(self, app: Paint, x, y):
66         if app.onCanvas(x, y):
67             app.buffer[y, x] = (255, 255, 255)
68
69 class Eyedropper(Tool):
70     def on_mouse_down(self, app: Paint, event):
71         app.colorBar.setPrimaryColor(
72             '#{02x}{02x}{02x}'.format(*
73             tuple(app.buffer[event.y, event.x]
74             )))
75
76 class Rectangle(Tool):
77     def __init__(self):
78         self.visual = 0
79
80     def on_mouse_down(self, app: Paint, event):
81         self.start = Point(event.x, event.y)
82
83     def on_mouse_drag(self, app: Paint, event):
84         self.stop = Point(event.x, event.y)
85         self.create_visual(app)
86
87     def on_mouse_up(self, app: Paint, event):
88         self.stop = Point(event.x, event.y)
89         app.rect(self.start, self.stop)
90         app.render()
91
92     def create_visual(self, app: Paint):
93         x1, x2 = sorted([self.start.x, self.stop.x])
94         y1, y2 = sorted([self.start.y, self.stop.y])

```

```

95         app.rect(Point(x1, y1), Point(x2, y2), temp = True)
96         app.render()
97
98     class Line(Tool):
99         def on_mouse_down(self, app: Paint, event):
100             self.last = Point(event.x, event.y)
101
102         def on_mouse_up(self, app: Paint, event):
103             app.line(self.last, Point(event.x, event.y))
104             app.render()
105
106         def on_mouse_drag(self, app: Paint, event):
107             app.line(self.last, Point(event.x, event.y), temp = True)
108             app.render()
109
110     class CustomShape(Tool):
111         def __init__(self):
112             self.points: list[Point] = []
113
114         def on_mouse_up(self, app: Paint, event):
115             if self.points:
116                 app.line(self.points[-1], Point(event.x, event.y))
117             self.points.append(Point(event.x, event.y))
118             app.render()
119
120         def on_mouse_move(self, app: Paint, event):
121             if self.points:
122                 self.create_visual(app, Point(event.x, event.y))
123
124         def on_enter(self, app: Paint):
125             app.line(self.points[-1], self.points[0])
126             self.points = []
127             app.clearTempBuffer()
128             app.render()
129
130         def create_visual(self, app: Paint, mousePos: Point):
131             app.line(self.points[-1], mousePos, temp = True)
132             app.render()
133
134     class Ellipse(Tool):
135         def __init__(self):
136             self.visual = 0
137
138         def on_mouse_down(self, app: Paint, event):
139             self.start = Point(event.x, event.y)
140
141         def on_mouse_drag(self, app: Paint, event):
142             self.stop = Point(event.x, event.y)
143             self.create_visual(app)

```

```
144
145 def on_mouse_up(self, app: Paint, event):
146     self.stop = Point(event.x, event.y)
147     app.ellipse(self.start, self.stop)
148
149 def create_visual(self, app: Paint):
150     app.ellipse(self.start, self.stop, temp = True)
```

actions.py

```
1 from collections import deque
2 from point import Point
3 import numpy as np
4
5 def rect(buffer, start: Point, stop: Point, color, fill = None, maskBuffer = None):
6
7     x1, x2 = sorted([start.x, stop.x])
8     y1, y2 = sorted([start.y, stop.y])
9
10    buffer[y1 : y2 + 1, x1 : x2 + 1] = color
11
12    if not isinstance(maskBuffer, type(None)):
13        maskBuffer[y1 : y2 + 1, x1 : x2 + 1] = True
14
15
16 def line(buffer, start: Point, stop: Point, color, maskBuffer = None):
17
18     steps = max(np.abs(np.array((start.x - stop.x, start.y - stop.y)))) + 1
19
20     # linear interpolation
21
22     points = np.round(
23         np.linspace(
24             (start.x, start.y),
25             (stop.x, stop.y),
26             steps
27         )).astype(int)
28
29     buffer[points[:, 1], points[:, 0]] = color
30
31     if not isinstance(maskBuffer, type(None)):
32         maskBuffer[points[:, 1], points[:, 0]] = True
33
34 def ellipse(buffer, start: Point, stop: Point, color, fill: bool, maskBuffer =
None):
35     x1, x2 = sorted([start.x, stop.x])
36     y1, y2 = sorted([start.y, stop.y])
37
38     rx = (x2 - x1) // 2
39     ry = (y2 - y1) // 2
40     cx = x1 + rx
41     cy = y1 + ry
42
43     rx2 = rx * rx
44     ry2 = ry * ry
45
46     x = 0
```

```

47     y = ry
48
49     dx = 2 * ry2 * x
50     dy = 2 * rx2 * y
51
52     def plot(px, py):
53         if fill:
54             # draw horizontal spans
55             for fx in range(cx - px, cx + px + 1):
56                 buffer[cy + py, fx] = color
57                 buffer[cy - py, fx] = color
58                 if not isinstance(maskBuffer, type(None)):
59                     maskBuffer[cy + py, fx] = True
60                     maskBuffer[cy - py, fx] = True
61         else:
62             points = [
63                 (cx + px, cy + py),
64                 (cx - px, cy + py),
65                 (cx + px, cy - py),
66                 (cx - px, cy - py),
67             ]
68             for px_, py_ in points:
69                 buffer[py_, px_] = color
70                 if not isinstance(maskBuffer, type(None)):
71                     maskBuffer[py_, px_] = True
72
73     # --- Region 1 ---
74     p1 = ry2 - rx2 * ry + 0.25 * rx2
75
76     while dx < dy:
77         plot(x, y)
78
79         if p1 < 0:
80             x += 1
81             dx += 2 * ry2
82             p1 += dx + ry2
83         else:
84             x += 1
85             y -= 1
86             dx += 2 * ry2
87             dy -= 2 * rx2
88             p1 += dx - dy + ry2
89
90     # --- Region 2 ---
91     p2 = (ry2 * (x + 0.5)**2 +
92           rx2 * (y - 1)**2 -
93           rx2 * ry2)
94
95     while y >= 0:

```



```

96         plot(x, y)
97
98     if p2 > 0:
99         y -= 1
100         dy -= 2 * rx2
101         p2 += rx2 - dy
102     else:
103         y -= 1
104         x += 1
105         dx += 2 * ry2
106         dy -= 2 * rx2
107         p2 += dx - dy + rx2
108
109 def floodFill(buffer, point: Point, color):
110
111     targetColor = tuple(buffer[point.y, point.x])
112
113     # Either current color, or if over network: provided color
114     newColor = color
115
116     if targetColor == newColor:
117         return
118
119     width, height, _ = buffer.shape
120
121     def onCanvas(x, y):
122         return 0 <= x < width and 0 <= y < height
123
124     queue = deque()
125     queue.append(point)
126
127     while queue:
128         x, y = queue.popleft()
129
130         if not onCanvas(x, y):
131             continue
132
133         if not tuple(buffer[y, x]) == targetColor:
134             continue
135
136         buffer[y, x] = newColor
137
138         queue.append((x - 1, y))
139         queue.append((x + 1, y))
140         queue.append((x, y + 1))
141         queue.append((x, y - 1))

```

point.py

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Point:
5     x: int
6     y: int
7
8     def __iter__(self):
9         yield self.x
10        yield self.y
```

main.py

```
1  from fastapi import FastAPI, WebSocket, WebSocketDisconnect
2  from dataclasses import dataclass
3  from typing import Optional
4
5  @dataclass
6  class Client:
7      id: int
8      name: Optional[str]
9      websocket: WebSocket
10
11  class ClientManager:
12      def __init__(self):
13          self.clients: list[Client] = []
14          self.nextId = 1
15
16      def removeConnection(self, client: Client):
17          self.clients.remove(client)
18
19      def addConnection(self, websocket: WebSocket) -> Client:
20          client = Client(
21              id = self.nextId,
22              name = None,
23              websocket = websocket
24          )
25
26          self.clients.append(client)
27          self.nextId += 1
28
29          return client
30
31      async def listen(self, client: Client):
32          try:
33              while True:
34                  data = await client.websocket.receive_json()
35                  await self.broadcast(data, client)
36
37          except WebSocketDisconnect:
38              self.removeConnection(client)
39
40      async def broadcast(self, data: str, exclusion: Client):
41          for client in self.clients:
42              if client != exclusion:
43                  await client.websocket.send_json(data)
44
45  app = FastAPI()
46  clientManager = ClientManager()
47
```

```
48 @app.websocket("/")
49 async def websocketEndpoint(websocket: WebSocket):
50     await websocket.accept()
51     client = clientManager.addConnection(websocket)
52
53     await clientManager.listen(client)
54
55 @app.get("/")
56 def get_clients():
57     return {
58         "clients": [
59             {
60                 "id": client.id,
61                 "name": client.name
62             }
63             for client in clientManager.clients
64         ]
65     }
```

network.py

```
1  from os import _exit
2  import websockets
3  import threading
4  import asyncio
5  from tkinter import messagebox
6  import queue
7  import json
8
9  class NetworkHandler:
10     def __init__(self, master, websocketURL: str):
11         self.wsURL: str = websocketURL
12         self.master = master
13
14         self.running: bool = False
15
16         self.queueIncoming = queue.Queue()
17         self.queueOutgoing = queue.Queue()
18
19         self.master.after(100, self.processIncoming)
20
21     def sendAction(self, action: dict):
22         if self.running:
23             self.queueOutgoing.put(json.dumps(action))
24
25     async def receiver(self, websocket: websockets.ClientConnection):
26         try:
27             while True:
28                 message = await websocket.recv()
29                 self.queueIncoming.put(message)
30
31         except websockets.ConnectionClosed:
32             print("Disconnecting")
33
34     async def sender(self, websocket: websockets.ClientConnection):
35         try:
36             while True:
37                 message = await asyncio.to_thread(self.queueOutgoing.get)
38                 await websocket.send(message)
39
40         except websockets.ConnectionClosed:
41             print("Disconnecting")
42
43     async def wsHandler(self):
44         for attempt in range(10):
45             try:
46                 async with websockets.connect(self.wsURL) as websocket:
47                     self.websocket = websocket
```

```

48
49         await asyncio.gather(
50             self.sender(websocket),
51             self.receiver(websocket)
52         )
53         return
54
55     except OSError as error:
56         print(f"Retry {attempt}: {error}")
57         await asyncio.sleep(3)
58
59     self.master.after(0, lambda: messagebox.showerror(
60         "Connection Error",
61         f"Failed to connect after retries:\n{self.wsURL}"
62     ))
63
64
65     def processIncoming(self):
66         while not self.queueIncoming.empty():
67             data: dict = json.loads(self.queueIncoming.get())
68             self.master.runAction(data)
69
70         self.master.after(100, self.processIncoming)
71
72     def runWsHandler(self):
73         self.asyncioLoop = asyncio.new_event_loop()
74         asyncio.set_event_loop(self.asyncioLoop)
75
76         self.asyncioLoop.run_until_complete(
77             self.wsHandler()
78         )
79
80     def stop(self):
81         if self.running:
82             if hasattr(self, "websocket"):
83                 self.asyncioLoop.call_soon_threadsafe(
84                     lambda: asyncio.create_task(
85                         self.websocket.close(code = 1000, reason = "Application
Closed")
86                     )
87                 )
88
89             _exit(0)
90
91     def run(self):
92         self.running = True
93
94         self.wsThread = threading.Thread(
95             target = self.runWsHandler,

```

```
96         daemon = True
97     )
98
99     self.wsThread.start()
100
```